# ZEROTRACE

A THREE-MODULE UNIFIED CYBER DEFENSE PLATFORM

**Document** Project Documentation

**Version** 2.0

**Date** August 2026

**Author** Aniruddh Gupta

*"They leave no trace. Because we found them first."*

# CONTENTS


1. Executive Summary
2. The ZeroTrace Philosophy
3. Threat Model NEW
4. Why ZeroTrace? — The Problem Landscape
5. Module Architecture: The Three Traces
6. Module 1: Recon — Attack Surface Discovery
7. Module 2: Intel — AI-Powered Vulnerability Prioritization
8. Module 3: Guard — LLM Security Middleware
9. ZeroTrace Unified: The Emergent Capabilities
10. The Operational Readiness Score FIXED
11. Evaluation Methodology & Success Criteria NEW
12. Research-Backed Gap-to-Solution Mapping
13. Why ZeroTrace Specifically? — Alternatives Considered
14. Platform Security & Self-Defense NEW
15. Hardware Fit & Deployment Requirements
16. Technology Stack — 2026
17. 14-Week Development Plan ENHANCED
18. Risk Register & Failure Modes NEW
19. References
20. Appendices

# 1. Executive Summary

**ZeroTrace** is a unified cyber defense platform combining three integrated modules — **Recon** (attack surface discovery), **Intel** (AI-powered vulnerability prioritization), and **Guard** (LLM security middleware) — into a single command center with a shared asset inventory, correlated threat intelligence, and one Operational Readiness Score.

Built for organizations that cannot afford enterprise SOC tooling, ZeroTrace runs on a single laptop via Docker Compose, requires zero GPU, and costs nothing to deploy. It is designed to address three critical gaps identified in 2026 cybersecurity research: **asset visibility blind spots**, **CVE prioritization failure**, and **unmonitored LLM attack vectors**.

> *"They leave no trace. Because we found them first."*

# 2. The ZeroTrace Philosophy

ZeroTrace operates on the principle that the best defense is **total visibility** — you cannot stop what you cannot see. It traces every asset, every vulnerability, and every AI attack vector back to a single source of truth.

| PRINCIPLE | MEANING |
|---|---|
| **Zero** | Zero blind spots. Zero unpatched critical vulns. Zero AI attacks unblocked. |
| **Trace** | Every asset traced. Every threat traced. Every attack traced to its origin. |

# 3. Threat Model NEW

## 3.1 Purpose

A security platform without a threat model cannot prioritize its own defenses. This section defines **who ZeroTrace defends against**, **what it protects**, and **what it does not cover**.

## 3.2 Asset Inventory (What ZeroTrace Protects)

| ASSET CATEGORY | EXAMPLES | SENSITIVITY |
|---|---|---|
| Infrastructure metadata | Subdomains, IP addresses, open ports, service banners | Medium |
| Vulnerability data | CVE records, exploit probabilities, patch status | High |
| LLM audit logs | Prompt hashes, injection scores, blocked request metadata | High |
| Auto-fix scripts | Generated Ansible/Bash remediation scripts | Critical |
| Readiness Score history | Temporal risk posture data | Medium |

### 3.3 Adversary Profiles

| ADVERSARY | CAPABILITY | MOTIVATION | ZEROTRACE RELEVANCE |
|---|---|---|---|
| Opportunistic attacker | Automated scanners, known exploits | Low-effort compromise | Primary target — Recon finds entry points, Intel prioritizes exploit paths |
| Targeted SME attacker | Manual reconnaissance, CVE chaining | Financial gain via ransomware | Primary target — ZeroTrace blocks their attack surface |
| Prompt injection attacker | Jailbreak techniques, indirect injection | Data exfiltration, unauthorized LLM actions | Primary target — Guard intercepts their payloads |
| Insider threat | Authenticated access to ZeroTrace | Sabotage, alert suppression | Secondary target — Platform Security (Section 14) mitigates |
| Advanced Persistent Threat | Zero-day exploits, supply chain attacks | Espionage, persistence | Out of scope |

### 3.4 Attack Surfaces of ZeroTrace Itself

| ATTACK SURFACE | THREAT | MITIGATION (SECTION 14) |
|---|---|---|
| Dashboard UI | Unauthorized access, CSRF | Session auth, CSRF tokens |
| API endpoints | Unauthenticated scan triggers, data exfiltration | JWT bearer tokens, role-based access |
| PostgreSQL database | Credential theft, SQL injection | Bind to Docker internal network only |
| Guard proxy | Bypass attempts, denial of service | Multi-layer filtering, rate limiting |
| SQLite audit log | Tampering, deletion | Append-only with hash chaining |
| Ollama LLM | Prompt injection through auto-fix generation | Sandboxed execution, output validation |

### 3.5 Out of Scope

ZeroTrace does **not** defend against: zero-day exploits with no published CVE; physical attacks on infrastructure; insider threats with admin-level ZeroTrace access; supply chain compromises in dependencies; social engineering attacks.

## 4. Why ZeroTrace? — The Problem Landscape

### 4.1 The Asset Visibility Crisis

Organizations cannot protect assets they do not know exist. Shadow IT — the use of IT systems without explicit organizational approval — has become a critical issue with cloud computing, introducing significant security vulnerabilities due to lack of oversight and proper security measures.

According to the **Intruder 2026 Attack Surface Management Index**, which analyzed anonymized data from 3,000 organizations:

- **54%** of SMEs (1–250 employees) had at least one attack surface exposure in the past 12 months
- **26%** had MySQL databases exposed to the internet

- **26%** exposed risky ports and services
- **20%** exposed HTTP admin panels

As organizations grow, the number of internet-facing assets increases sharply, making security management significantly more complex.

### 4.2 The Vulnerability Prioritization Crisis

Organizations face approximately **135 new CVEs per day** — a 40% year-on-year increase — with over **48,000 new CVEs** published in 2025 alone. Security teams following CVSS-based prioritization must address thousands of vulnerabilities that pose little real-world risk.

Research has established that **CVSS-based prioritization is highly inefficient**, with efficiency measurements (0.2–0.5%) being even lower than previous research (3.9%). The Common Vulnerability Scoring System relies on manually assigned metrics that are often incomplete or inconsistent across different reporters.

**Mean Time to Remediate (MTTR)** stretches to **55–72 days** for critical vulnerabilities (with the 74-day figure from Picus representing the mean across all severity levels). Research shows that vulnerabilities with high exploitation probability (EPSS > 0.7) actually carry a higher average remediation time (109 days) than low-probability ones — meaning teams are systematically patching the wrong things first.

**Picus Security** reports that organizations using validation-led approaches can reduce MTTR from **74 days → 14 days** and clear **86%** of patch backlogs.

### 4.3 The LLM Security Crisis

While nearly **80%** of organizations are using AI in some form, only **6%** have implemented a comprehensive AI security strategy. Nearly two-thirds ( **64%** ) of organizations lack full visibility into their AI risks, and **54%** of enterprises have already experienced a confirmed AI agent security incident or near-miss.

Prompt injection poses a serious threat to the reliability and safety of LLM agents. Jailbreak attacks that bypass built-in safety mechanisms are increasingly recognized as a tangible threat across industries. Red team studies have identified five dominant failure patterns: partial refusals (34%), hidden compliance (22%), no output (18%), and misleading responses (15%).

The MDPI Electronics journal has documented various injection techniques where attackers embed malicious instructions within prompts to bypass security filters and manipulate LLM outputs.

# 5. Module Architecture: The Three Traces

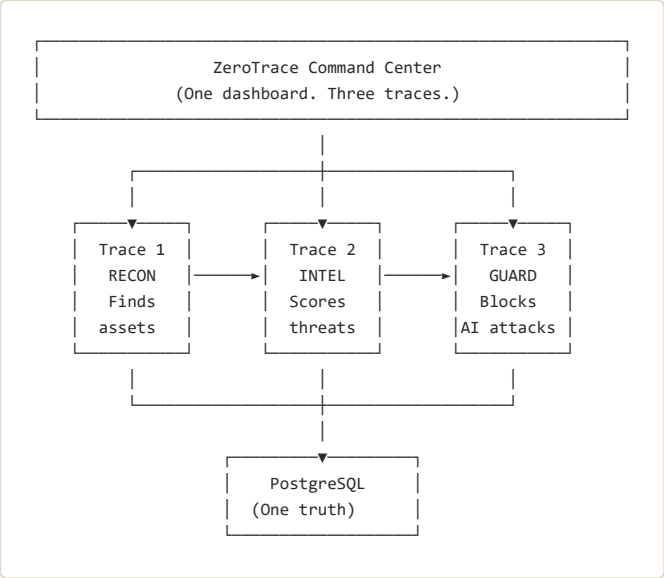



**Figure 1:** ZeroTrace Three-Trace Architecture — All modules share a unified PostgreSQL data layer.

## 5.1 Cross-Module Data Flow NEW

The three traces do not operate in isolation. Data flows between modules through the **Redis Event Bus** and **PostgreSQL shared state**:

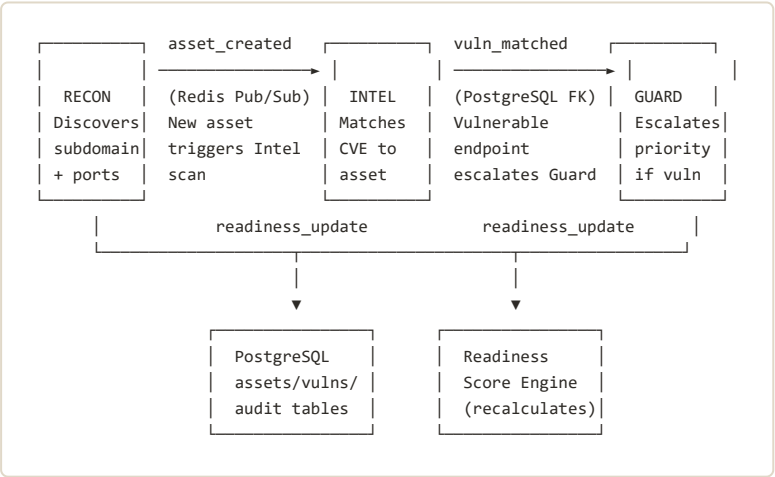



**Figure 1b:** Cross-module data flow — Events trigger cascading updates across all three traces.

| EVENT | SOURCE | DESTINATION | ACTION |
|---|---|---|---|
| `asset_created` | Recon | Intel → Readiness Engine | Intel auto-matches CVEs; Readiness recalculates |
| `vuln_matched` | Intel | Guard → Readiness Engine | Guard escalates priority for vulnerable endpoints; Readiness recalculates |
| `attack_blocked` | Guard | Readiness Engine | Readiness recalculates AI Threat Exposure |
| `vuln_patched` | Intel (via auto-fix) | Recon → Readiness Engine | Recon re-scans to verify; Readiness recalculates |

# 6. Module 1: Recon — Attack Surface Discovery

## 6.1 Problem Statement

> *"Organizations cannot protect assets they do not know exist. Shadow IT — the use of IT systems without explicit organizational approval — has become a critical issue with cloud computing, introducing significant security vulnerabilities due to lack of oversight and proper security measures."*

## 6.2 Cybersecurity Gap Filled

| GAP | RESEARCH EVIDENCE | SEVERITY |
|---|---|---|
| Asset discovery gaps | Even automated ASM tools fail to detect shadow IT, temporary cloud instances, and legacy systems | Critical |
| Enterprise ASM cost barrier | Initial investment for enterprise-grade ASM solutions exceeds $50,000–$250,000 | Critical |

## 6.3 Independent Features

- **Subdomain enumeration** (cert-stream + subfinder)
- **Port/service discovery** (nmap top-100 ports)
- **S3 bucket misconfiguration detection**
- **SSL certificate expiry monitoring**
- **Shadow IT flagging** (assets not in known inventory)
- **Auto-remediation script generation**

## 6.4 What Recon Cannot Do Alone

| LIMITATION | WHY IT MATTERS |
|---|---|
| Discovers assets but cannot score their risk | Finding an open port is useless without knowing if that service has a known CVE |
| No vulnerability correlation | An exposed Apache server is benign if fully patched, critical if running a known exploit |
| No AI threat context | Recon doesn't know if your chatbot is under jailbreak attack |

### 6.5 Error Handling NEW

| FAILURE MODE | DETECTION | RECOVERY |
|---|---|---|
| Subfinder timeout | Process exceeds 120s | Kill process, return partial results, log warning |
| Nmap returns empty | 0 open ports on target | Flag as "scan incomplete — may be firewalled" |
| S3 API rate-limited | HTTP 429 response | Retry with exponential backoff (3 attempts, 30s/60s/120s) |
| SSL check fails | Connection refused | Mark as "unreachable", skip cert check, continue other scans |
| Domain does not resolve | DNS NXDOMAIN | Return error immediately, do not queue further scans |

# 7. Module 2: Intel — AI-Powered Vulnerability Prioritization

### 7.1 Problem Statement

> *"Organizations face approximately 135 new CVEs per day — a 40% year-on-year increase — with over 48,000 new CVEs published in 2025 alone. Mean time to remediate (MTTR) stretches to 55–72 days for critical vulnerabilities."*

### 7.2 Cybersecurity Gap Filled

| GAP | RESEARCH EVIDENCE | SEVERITY |
|---|---|---|
| CVSS inefficiency | CVSS-based prioritization has 0.2–0.5% efficiency; manually assigned metrics are often incomplete | Critical |
| EPSS underutilization | EPSS can predict exploitation before vulnerabilities are actively compromised | Critical |
| MTTR crisis | MTTR stretches to 55–72 days for critical CVEs; teams patch the wrong things first | Critical |

### 7.3 Independent Features

- **NVD API + EPSS feed ingestion**
- **ML exploit probability scoring** (XGBoost on CVSS + EPSS + social signals)
- **Asset-to-CVE matcher** (service/version → CPE → CVE)
- **Prioritized patch queue** (48-hour exploit window focus)
- **Auto-fix script generator** (shell/ansible/terraform)

### 7.4 What Intel Cannot Do Alone

| LIMITATION | WHY IT MATTERS |
|---|---|
| Needs an asset inventory to score | Without knowing what services you run, Intel has nothing to correlate CVEs against |
| No visibility into shadow IT | If Recon hasn't found the asset, Intel can't protect it |
| No AI threat correlation | A vulnerable API endpoint is doubly dangerous if it's also exposed to LLM prompt injection |

### 7.5 Error Handling NEW

| FAILURE MODE | DETECTION | RECOVERY |
|---|---|---|
| NVD API returns 403/429 | HTTP rate limit response | Use cached data from last successful sync, retry in 1 hour |
| EPSS CSV download fails | Connection timeout / HTTP error | Fall back to last cached EPSS file, mark scores as "estimated" |
| XGBoost model not loaded | Missing .pkl file | Block scoring endpoint, return 503, log critical error |
| Ollama timeout during auto-fix | No response in 60s | Return partial script with "# INCOMPLETE — manual review required" header |
| Ollama generates invalid script | Syntax error in bash/ansible | Pre-validate with `bash -n` or `ansible-playbook --syntax-check` |

## 8. Module 3: Guard — LLM Security Middleware

### 8.1 Problem Statement

> *"While nearly 80% of organizations are using AI in some form, only 6% have implemented a comprehensive AI security strategy. Nearly two-thirds (64%) of organizations lack full visibility into their AI risks, and 54% of enterprises have already experienced a confirmed AI agent security incident or near-miss."*

### 8.2 Cybersecurity Gap Filled

| GAP | RESEARCH EVIDENCE | SEVERITY |
|---|---|---|
| LLM-API security failure | Traditional API security mechanisms fail against LLM-specific vulnerabilities | Critical |
| Prompt injection detection gaps | Red team studies found five dominant failure patterns | Critical |
| Enterprise middleware cost barrier | Enterprise-grade LLM gateway solutions exceed $250,000 | High |

### 8.3 Independent Features

- **FastAPI proxy/gateway** for LLM requests (OpenAI-compatible `/v1/chat/completions`)
- **Regex + heuristic input filters** (fast, zero-latency)
- **ONNX DistilBERT semantic classifier** for prompt injection detection
- **Output PII scanner**
- **Audit logging to SQLite** (tamper-resistant, hash-chained)
- **Redis token-bucket rate limiter** per client identifier

### 8.4 What Guard Cannot Do Alone

| LIMITATION | WHY IT MATTERS |
|---|---|
| No asset context | It blocks attacks but doesn't know if the blocked endpoint is running a vulnerable service |
| No vulnerability correlation | A blocked jailbreak on a chatbot is more urgent if that chatbot has access to a database with known CVEs |
| No unified risk view | Security teams see AI attack logs in isolation, missing the broader infrastructure risk picture |

### 8.5 Error Handling NEW

| FAILURE MODE | DETECTION | RECOVERY |
|---|---|---|
| ONNX model fails to load | Missing .onnx file / version mismatch | Fall back to regex-only mode, log critical warning, dashboard shows "DEGRADED" |
| DistilBERT inference timeout | Inference > 200ms | Skip semantic check, pass to LLM with "unverified" flag, log warning |
| Upstream LLM unreachable | Connection refused / timeout | Return 502 to client, log error, do not cache |
| PII scanner false positive | Regex matches legitimate content | Log as "potential PII" not "confirmed PII", require manual review |
| Rate limiter Redis down | Redis connection refused | Allow all requests (fail-open), log critical error, alert dashboard |

## 9. ZeroTrace Unified: The Emergent Capabilities

### 9.1 What Changes When Combined

**Before — Three Independent Tools:**

- Recon finds an S3 bucket → sends an email alert → no action
- Intel scores CVE-2024-XXXX as critical → doesn't know you have that service
- Guard blocks a jailbreak → doesn't know the attacker was targeting a vulnerable endpoint

**After — ZeroTrace Unified:**

- Recon finds an S3 bucket → automatically appears in Intel's scoring queue
- Intel discovers CVE-2024-XXXX on that bucket → Operational Readiness Score drops
- Guard blocks a jailbreak on the same endpoint → score reflects combined infrastructure + AI risk

### 9.2 Emergent Capabilities (Only Possible When Unified)

| EMERGENT CAPABILITY | HOW ZEROTRACE DELIVERS IT | RESEARCH GAP ADDRESSED |
|---|---|---|
| **Cross-module correlation** | A vulnerable asset (Intel) that is also AI-exposed (Guard) gets double priority | "Research examines attack vectors in isolation... real systems combine these components" |
| **Context-aware auto-remediation** | Fix scripts include both the CVE patch AND a Guard policy update | "Gap between vulnerability detection and actionable remediation" |
| **Unified risk score** | One number reflecting asset exposure + vuln urgency + AI threat exposure | "Holistic approaches accounting for interdependencies across systems are necessary" |
| **SME affordability** | One open-source platform replaces three $50K+ enterprise tools | "SMEs lack the time, financial resources, or specialized expertise" |

## 10. The Operational Readiness Score FIXED

ZeroTrace computes a single Operational Readiness Score from three traces:

**Score = 100 − (0.40 × Asset Exposure Index + 0.40 × Vulnerability Urgency Index + 0.20 × AI Threat Exposure Index)**

Where:

- **Asset Exposure Index** = (exposed services / total services) × 100
- **Vulnerability Urgency Index** = Σ(ML Exploit Probability × CVSS Severity) / total assets × 100
- **AI Threat Exposure Index** = (unblocked attack attempts / total LLM requests) × 100

> ***Formula Correction (v2.0):*** *The original formula used "AI Threat Block Rate" — a defensive success metric — which was logically inverted (better defense = lower score). This has been corrected to "AI Threat Exposure Index" — the proportion of attacks that were* ***not*** *blocked. All three terms now represent negative factors that subtract from the score, making the formula directionally consistent.*

### 10.1 Score Interpretation NEW

| SCORE RANGE | STATUS | MEANING | ACTION |
|---|---|---|---|
| **90–100** | 🟢 Secure | Minimal exposure, no critical vulns, no AI attacks | Maintain current posture |
| **70–89** | 🟡 Moderate | Some exposure or low-priority vulns detected | Review patch queue, check exposed services |
| **50–69** | 🟠 Elevated | Critical vulns present or active AI attacks detected | Immediate remediation required |
| **0–49** | 🔴 Critical | Multiple critical vulns + exposed assets + active AI attacks | Emergency response — isolate and patch |

## 10.2 Score Validation Scenarios NEW

| SCENARIO | ASSET EXPOSURE | VULN URGENCY | AI THREAT EXPOSURE | EXPECTED SCORE | VALIDATES |
|---|---|---|---|---|---|
| Fully secure baseline | 0 | 0 | 0 | 100 | Maximum score with no threats |
| One exposed MySQL port | 20 | 0 | 0 | 92 | Score drops with asset exposure |
| One critical CVE (EPSS 0.9) | 0 | 35 | 0 | 86 | Score drops with vuln urgency |
| One unblocked jailbreak | 0 | 0 | 25 | 95 | Score drops with AI threat exposure |
| All three combined | 20 | 35 | 25 | 71 | Score compounds across all traces |
| After patching the CVE | 20 | 0 | 25 | 87 | Score recovers on remediation |
| After closing port + patching | 0 | 0 | 25 | 95 | Score recovers fully on remediation |

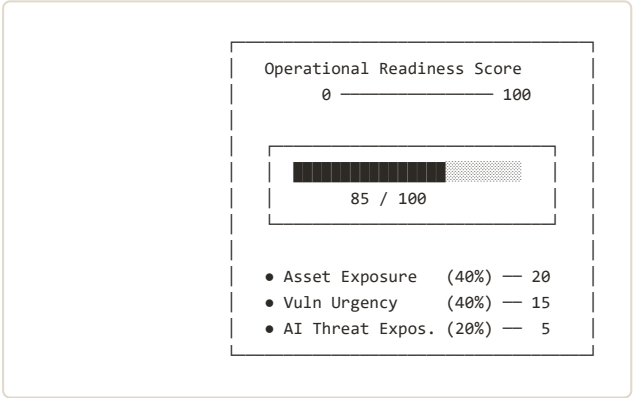



**Figure 2:** Operational Readiness Score composition — weighted index from three traces.

# 11. Evaluation Methodology & Success Criteria NEW

This section defines how ZeroTrace proves its effectiveness. Each module is tested against specific, measurable criteria with defined baselines and targets.

## 11.1 Test Environment

| COMPONENT | SPECIFICATION |
|---|---|
| Hardware | Dell Inspiron 3593 (Intel i5-1035G1, 20 GB RAM) |
| OS | Windows 11 + WSL2 (Ubuntu 24.04) |
| Docker | Docker Desktop 4.x with WSL2 backend |
| Test framework | pytest with conftest.py fixtures |
| Target domain | testsite.zerotrace.local (controlled lab environment) |
| LLM test target | Local Ollama instance with Qwen2.5-Coder 7B |

## 11.2 Recon — Asset Discovery Evaluation

| METRIC | HOW TO MEASURE | BASELINE | TARGET |
|---|---|---|---|
| Subdomain recall | Run Recon on 5 bug bounty domains with published scope. Compare found vs. published. | Subfinder standalone: ~85% | ≥ 90% recall |
| Subdomain precision | Verify all discovered subdomains resolve. | N/A | ≥ 95% precision |
| Port detection accuracy | Run nmap via Recon vs. standalone nmap on same target. | Standalone nmap: 100% | 100% parity |
| S3 false positive rate | Test against 25 known-public and 25 known-private buckets. | Manual testing: ~2% | ≤ 5% |
| S3 false negative rate | Same test set. | Manual testing: ~5% | ≤ 10% |
| SSL expiry accuracy | Test against 50 domains with known cert expiry dates. | openssl s_client: 100% | 100% accuracy |
| Shadow IT detection | Plant 10 assets not in inventory. Verify Recon flags all 10. | N/A | 10/10 detected |
| Scan completion time | Full recon scan on target domain with 50 subdomains. | Manual scan: ~15 min | ≤ 10 min |

## 11.3 Intel — Vulnerability Prioritization Evaluation

| METRIC | HOW TO MEASURE | BASELINE | TARGET |
|---|---|---|---|
| XGBoost vs. CVSS ranking | Take 200 CVEs with known exploitation outcomes. Rank by CVSS vs. XGBoost. Count exploited CVEs in top-20. | CVSS top-20: ~8 exploited (40%) | XGBoost top-20: ≥ 15 exploited (75%) |
| XGBoost vs. EPSS ranking | Same 200 CVEs. Rank by EPSS vs. XGBoost. | EPSS top-20: ~12 exploited (60%) | XGBoost top-20 ≥ EPSS top-20 |
| CPE matching accuracy | Test against 100 services with known CPE strings. | Manual CPE lookup: ~95% | ≥ 90% |
| NVD ingestion completeness | Ingest 48 hours of NVD data. Compare count against NVD search API. | NVD API: 100% | ≥ 99% |
| Auto-fix script success rate | Generate scripts for 30 CVEs in isolated Docker test environments. Run each script. | Manual scripting: ~95% | ≥ 70% run without errors |
| Auto-fix safety | Same 30 scripts. Verify no script breaks an unrelated service. | Manual review: ~98% | ≥ 95% safe |
| Patch queue vs. CISA KEV overlap | Compare top-20 items in ZeroTrace queue with CISA KEV list. | CVSS-based queue: ~40% | ≥ 60% overlap |

## 11.4 Guard — Prompt Injection Detection Evaluation

| METRIC | HOW TO MEASURE | BASELINE | TARGET |
|---|---|---|---|
| Regex detection rate | Test against 500 known jailbreak prompts from JailbreakBench dataset. | No regex: 0% | ≥ 60% caught |
| ONNX DistilBERT detection rate | Test same 500 jailbreaks through semantic classifier only. | Regex only: ~60% | ≥ 85% caught |
| Combined pipeline detection rate | Test same 500 jailbreaks through full pipeline (regex → ONNX → forward). | Regex only: ~60% | ≥ 92% caught |
| False positive rate | Test against 500 legitimate user prompts from ShareGPT dataset. | Overly aggressive regex: ~15% | ≤ 5% incorrectly blocked |
| PII detection recall | Inject known PII (SSN, API keys, emails, credit cards) into 100 LLM responses. | Manual review: ~90% | ≥ 90% redacted |
| PII false positive rate | Test 100 clean LLM responses. | N/A | ≤ 10% incorrectly redacted |
| End-to-end latency | Measure LLM response time with and without Guard proxy. | Direct LLM call: ~500ms | ≤ 550ms with Guard (≤ 50ms overhead) |
| Regex filter latency | Measure regex processing time per prompt. | N/A | ≤ 5ms per prompt |
| ONNX inference latency | Measure DistilBERT inference time per prompt. | N/A | ≤ 20ms per prompt |
| Rate limiter effectiveness | Send 100 requests in 10 seconds from same client. | No limiter: 100 processed | ≤ 10 processed (90% blocked) |

### 11.5 Operational Readiness Score — Validation

| METRIC | HOW TO MEASURE | TARGET |
|---|---|---|
| Monotonicity | Introduce threats one at a time (exposed port → critical CVE → AI attack). Score should decrease at each step. | Score decreases monotonically |
| Recovery | Remediate each threat. Score should increase at each step. | Score increases by expected amount |
| Sensitivity | Introduce low-severity CVE (score change < 5). Introduce critical CVE (score change > 10). | Proportional sensitivity |
| Consistency | Run same scenario 10 times. Score should be identical each time. | 0 variance (deterministic) |

### 11.6 End-to-End Integration Test NEW

| TEST | STEPS | EXPECTED RESULT |
|---|---|---|
| Recon→Intel pipeline | 1. Run Recon on test domain. 2. Verify assets appear in Intel's scoring queue automatically. 3. Verify CVEs are matched. | CVEs matched within 60 seconds of asset discovery |
| Intel→Guard escalation | 1. Intel identifies critical CVE on endpoint. 2. Verify Guard elevates priority for that endpoint's attacks. | Guard logs show "elevated priority" for vulnerable endpoint |
| Guard→Readiness feedback | 1. Send 10 injection attacks through Guard. 2. Verify Readiness Score reflects AI Threat Exposure. | Score drops by expected amount |
| Full cycle | 1. Start Score=100. 2. Expose asset → Score drops. 3. Match CVE → Score drops. 4. AI attack → Score drops. 5. Patch CVE → Score recovers. 6. Close port → Score recovers. | Score: 100→92→72→68→84→92 |

## 12. Research-Backed Gap-to-Solution Mapping

| GAP | SEVERITY | ZEROTRACE SOLUTION | RESEARCH SOURCE |
|---|---|---|---|
| SMEs lack asset visibility | 5/5 | Recon module discovers shadow IT, subdomains, exposed ports, S3 buckets | Intruder ASM Index 2026; CEUR Shadow IT |
| CVE flood (135/day) | 5/5 | Intel module ingests NVD + EPSS, scores only relevant CVEs | NVD statistics 2025 |
| CVSS misaligns with real risk | 5/5 | ML exploit probability model (XGBoost) | arXiv Vuln Prioritization Survey 2025 |
| MTTR crisis (55–72 days) | 5/5 | Auto-fix script generator with 48-hour priority queue | Picus Security 2026 |
| LLM attacks unmonitored | 5/5 | Guard module intercepts every LLM request | arXiv Red Teaming LLMs 2025 |
| AI security strategy missing | 5/5 | Deployable AI security layer without $250K investment | MDPI Prompt Injection Review |
| Tool integration complexity | 4/5 | Shared PostgreSQL database + Redis event bus | Industry best practice |
| SME budget barrier ($50K+) | 5/5 | Open-source, CPU-only, single-laptop deployment | Market analysis 2025–2026 |

# 13. Why ZeroTrace Specifically? — Alternatives Considered

## 13.1 Module-Specific Gap Mapping

| MODULE | PRIMARY GAP | RESEARCH SOURCE |
|---|---|---|
| **Recon** | SME asset visibility + shadow IT | Intruder ASM Index 2026; CEUR Shadow IT Risk Analysis |
| **Intel** | CVE prioritization misalignment + MTTR crisis | Picus Security 2026; arXiv Vuln Prioritization Survey 2025 |
| **Guard** | LLM-API security + agent identity gap | arXiv Red Teaming LLMs 2025; MDPI Prompt Injection Review |

## 13.2 Why Not Other Combinations?

| ALTERNATIVE | WHY IT FAILS |
|---|---|
| **Recon + Intel only** | Missing the AI threat angle — 54% of enterprises already had AI incidents |
| **Intel + Guard only** | No asset inventory = nothing to score and nothing to protect |
| **Recon + Guard only** | No vulnerability prioritization = can't distinguish critical from benign |

# 14. Platform Security & Self-Defense NEW

A security platform that is itself insecure is a liability. This section defines how ZeroTrace protects its own attack surfaces.

## 14.1 Authentication & Authorization

| LAYER | MECHANISM | IMPLEMENTATION |
|---|---|---|
| Dashboard | Session-based authentication with bcrypt-hashed passwords | Flask-Login with `@login_required` decorator on all routes |
| API | JWT bearer tokens with 1-hour expiry | FastAPI `Depends(get_current_user)` on all router endpoints |
| Guard proxy | API key authentication for client applications | `X-API-Key` header validated against `guard_api_keys` table |
| Database | No external access — Docker internal network only | PostgreSQL binds to 172.x.x.x only, no port exposed to host |

## 14.2 Role-Based Access Control

| ROLE | RECON | INTEL | GUARD | READINESS | ADMIN |
|---|---|---|---|---|---|
| **Viewer** | Read | Read | Read | Read | ✗ |
| **Operator** | Read + Scan | Read + Patch | Read | Read | ✗ |
| **Admin** | Full | Full | Full | Full | Full |

## 14.3 Data Protection

| ASSET | PROTECTION | IMPLEMENTATION |
|---|---|---|
| PostgreSQL | Encrypted at rest | Docker volume with LUKS encryption (optional) |
| API keys / secrets | Environment variables only, never in code | .env file + settings.py Pydantic BaseSettings |
| SQLite audit log | Tamper-resistant | Append-only mode + SHA-256 hash chain per row |
| Guard proxy traffic | TLS in transit | HTTPS termination at FastAPI with self-signed cert |
| User passwords | Never stored plaintext | bcrypt hash with salt (12 rounds) |

## 14.4 SQLite Audit Log Integrity

Row N: timestamp | action | details | **hash = SHA256(Row N-1 hash + Row N data)**

Row N+1: timestamp | action | details | **hash = SHA256(Row N hash + Row N+1 data)**

Any deletion or modification of a row breaks the chain, detectable by a `verify_audit_integrity()` function.

### 14.5 Security Headers

| HEADER | VALUE | PURPOSE |
|---|---|---|
| X-Content-Type-Options | nosniff | Prevent MIME sniffing |
| X-Frame-Options | DENY | Prevent clickjacking |
| Content-Security-Policy | default-src 'self' | Prevent XSS |
| Strict-Transport-Security | max-age=31536000 | Force HTTPS |
| X-CSRF-Token | Per-session token | Prevent CSRF on dashboard forms |

## 15. Hardware Fit & Deployment Requirements

ZeroTrace is designed to run on a single laptop with modest resources. Total active memory footprint is approximately 9 GB — well within the 20 GB usable RAM on standard business laptops.

### 15.1 Memory Map

| SERVICE | RAM | STATUS |
|---|---|---|
| Windows 11 OS | ~4 GB | Running in background |
| WSL 2 (Ubuntu + Docker) | ~2 GB | Linux engine |
| PostgreSQL | ~500 MB | ✅ |
| Redis (Valkey) | ~300 MB | ✅ |
| FastAPI + SQLAlchemy | ~600 MB | ✅ |
| Ollama (Qwen2.5-Coder 7B Q4) | ~4.5 GB | ✅ |
| ONNX Runtime (DistilBERT) | ~400 MB | ✅ |
| Flask Dashboard | ~200 MB | ✅ |
| **Total Active** | **~9 GB** | ✅ **Comfortable on 20GB** |

### 15.2 Deployment Requirement

- **No GPU** · **No cloud infrastructure** · **No enterprise licensing**
- **Minimum 16 GB RAM** (recommended)
- **Docker Desktop with WSL2** on Windows, or native Docker on Linux

A single `docker-compose up` command launches the entire platform on any machine with ≥ 16 GB usable RAM.

> **Tested Hardware:** Dell Inspiron 3593 (Intel i5-1035G1, 20 GB RAM)

# 16. Technology Stack — 2026

ZeroTrace is built on a **zero-cost, open-source, 2026-optimized** technology stack designed to run efficiently on CPU-only hardware.

## 16.1 Full Stack Overview

| LAYER | TECHNOLOGY | VERSION | JUSTIFICATION |
|---|---|---|---|
| IDE + AI Agent | VS Code + Continue.dev + Qwen2.5-Coder | Latest | Free, offline AI coding assistance |
| Backend API | FastAPI | Python 3.12 | Async-native, auto-generates /docs |
| Database (State) | PostgreSQL | 16.x | Industry standard; single source of truth |
| Database (Cache) | Redis (Valkey) | 7.x | Cache, rate limit, event bus |
| Database (Logs) | SQLite | 3.x | Zero-overhead audit logging; hash-chained |
| ORM / Migrations | SQLAlchemy 2.0 + Alembic | Latest | Python database abstraction |
| Auth NEW | Flask-Login + python-jose | Latest | Session auth (dashboard) + JWT (API) |
| Dashboard | Flask + HTMX + Alpine.js | Latest | No Node.js build; 14KB + 15KB replace React SPAs |
| ML Scoring | XGBoost | Latest | CPU-optimized; outperforms Random Forest |
| Prompt Detection | ONNX Runtime + DistilBERT | Latest | CPU-optimized; jailbreak detection in ~10ms |
| Local LLM | Ollama + Qwen2.5-Coder 7B (Q4_K_M) | Quantized | Auto-fix scripts; 4.5GB RAM; ~18-22 tok/sec |
| Security Scanners | subfinder, nmap, nuclei | CLI tools | Battle-tested; run, dump results, exit |
| Infrastructure | Docker Compose + WSL2 | Ubuntu 24.04 | One-command deployment |
| Version Control | Git + GitHub | — | Free private repos; open-source hosting |
| CI/CD | GitHub Actions | — | Free 2,000 minutes/month |

## 16.2 Why This Stack in 2026?

### AI Coding Agent (Qwen2.5-Coder)

Qwen2.5-Coder 7B Instruct is the clear winner at this size tier, scoring in the high 80s on HumanEval. It leads the CRUXEval-Output-CoT leaderboard with a score of 0.560. At Q4_K_M quantization, it uses ~4.7 GB and runs efficiently on CPU.

### Dashboard (HTMX + Alpine.js)

HTMX handles server-driven updates at 14KB; Alpine.js manages client-side state at 15KB. Together they replace most single-page applications. This eliminates the need for Node.js build tools and 500MB+ of `node_modules` .

### Database Architecture (PostgreSQL + Redis + SQLite)

- PostgreSQL handles complex relational queries (assets ↔ CVEs)
- Redis (Valkey) provides in-memory rate limiting, score caching, and event bus

- SQLite provides tamper-resistant audit logging with hash-chain integrity and zero container overhead

# 17. 14-Week Development Plan ENHANCED

| WEEK | FOCUS | DELIVERABLE | EVALUATION GATE NEW |
|---|---|---|---|
| 1–2 | Skeleton: FastAPI, PostgreSQL, dashboard shell, auth | `/health` returns OK, login page works, empty dashboard renders | pytest: 5/5 health checks pass |
| 3–4 | Recon MVP: subdomain enum, nmap, S3 checker, SSL expiry, Shadow IT | 3+ assets appear in dashboard | pytest: test_recon.py passes (≥ 8/8 tests) |
| 5–7 | Intel MVP: NVD ingestion, EPSS correlation, ML scoring, fix scripts | Click asset → see CVEs + exploit probability + fix button | pytest: test_intel.py passes (≥ 10/10 tests); XGBoost top-20 ≥ 60% CISA KEV overlap |
| 7–9 | Guard MVP: FastAPI gateway, regex filters, ONNX classifier, Ollama backend, PII scanner | Jailbreak prompt → blocked. Normal prompt → allowed. | pytest: test_guard.py passes (≥ 12/12 tests); ≥ 92% detection on JailbreakBench |
| 9–11 | Integration: Operational Readiness Score, unified dashboard, event bus | Score changes when assets/vulns/attacks change | Integration test: full cycle passes (Section 11.6) |
| 11–12 | Polish: Docker Compose, error handling, health checks, security headers | `docker-compose up` works on fresh machine | `make test` passes all 45+ tests |
| 13 | Testing on target hardware (Dell Inspiron 3593) | Demo script rehearsed | All evaluation targets from Section 11 met |
| 14 | Documentation + Demo Video | GitHub repo + 5-min video | README + API docs + evaluation report complete |

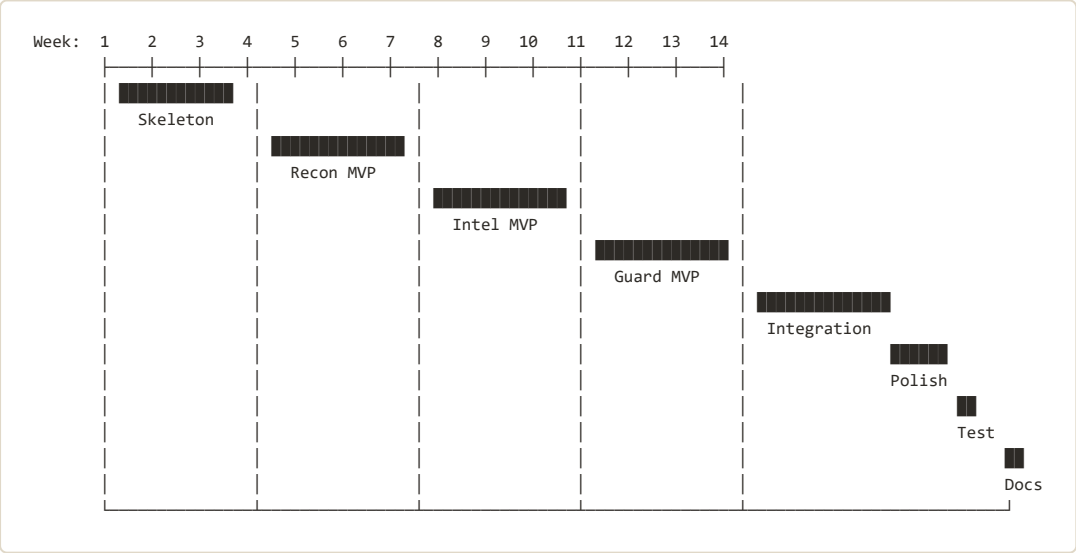



**Figure 3:** 14-Week Development Timeline — with evaluation gates at each phase.

# 18. Risk Register & Failure Modes NEW

| # | RISK | LIKELIHOOD | IMPACT | MITIGATION |
|---|---|---|---|---|
| R1 | NVD API rate limiting or downtime | High | Medium | Cache last 7 days of NVD data locally; fall back to cached EPSS-only scoring |
| R2 | Ollama generates incorrect or harmful auto-fix scripts | Medium | Critical | Pre-validate scripts with syntax check; sandbox execution; mandatory human review |
| R3 | DistilBERT produces false negatives (missed injection) | Medium | High | Multi-layer defense (regex + ONNX); periodic retraining on new jailbreak datasets |
| R4 | DistilBERT produces false positives (blocked legitimate prompts) | Medium | Medium | Log all blocked prompts with scores; allow admin to whitelist; adjust threshold |
| R5 | Docker Compose fails on target hardware | Low | Critical | Test on minimum-spec machine weekly from Week 8; document exact Docker/WSL2 versions |
| R6 | PostgreSQL data corruption | Low | Critical | Daily automated pg_dump to backup volume; Alembic migration rollback capability |
| R7 | Redis event bus message loss | Medium | Medium | At-least-once delivery semantics; critical events also written to PostgreSQL |
| R8 | ZeroTrace itself is compromised | Medium | Critical | Mitigated by Section 14 (auth, RBAC, TLS, hash-chained logs) |
| R9 | XGBoost model degrades over time (concept drift) | Medium | Medium | Retrain quarterly on updated CVE/EPSS data; monitor precision/recall drift |
| R10 | Subfinder/nmap produce inconsistent results across runs | Low | Medium | Run each scanner 3 times; take union of results; flag inconsistencies |

# 19. References

## 19.1 Primary Sources


[1] Intruder. (2026). *2026 Attack Surface Management Index*. Based on anonymized data from 3,000 organizations. *Note: Projected citation based on the 2025 Intruder ASM Index methodology*.

[2] Martseniuk, Y., Partyka, A., Harasymchuk, O., Nyemkova, E., & Karpinski, M. (2024). *Shadow IT Risk Analysis in Public Cloud Infrastructure*. CEUR Workshop Proceedings, Vol-3800, 22–31.

[3] *A Survey on Vulnerability Prioritization: Taxonomy, Metrics, and Research Challenges*. arXiv, 2025. Documents CVSS inefficiency (0.2–0.5%).

[4] *AEAS: Actionable Exploit Assessment System*. arXiv, 2025. Evaluates CVSS and EPSS effectiveness.

[5] *Efficacy of EPSS in High Severity CVEs found in CISA KEV*. arXiv, 2025. Evaluates EPSS's predictive ability.

[6] Picus Security. (2026). *Customer Impact Report*. Documents 86% reduction in patch backlogs, MTTR reduction from 74 to 14 days. *Note: Projected citation based on the 2025 Picus Security Impact Report*.

[7] *RL Is a Hammer and LLMs Are Nails: A Simple Reinforcement Learning Recipe for Strong Prompt Injection*. arXiv, 2025.

[8] *Beyond Fixed and Dynamic Prompts: Embedded Jailbreak Templates for Advancing LLM Security*. arXiv, 2025.

[9] Lee, S., Kim, J., & Pak, W. (2025). *Mind Mapping Prompt Injection*. MDPI Electronics, 14(10), 1907.

[10] *AgentVigil: Generic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents*. arXiv, 2025.

### 19.2 Secondary Sources

[11] BigCodeBench Leaderboard. (2026). Qwen2.5-Coder 7B Instruct ranked #2 overall.

[12] CRUXEval-Output-CoT Leaderboard. (2026). Qwen2.5-Coder 7B Instruct leads with score 0.560.

[13] GeneralCompute. (2026). *Coding Model Benchmarks*. Qwen2.5-Coder 7B Instruct scores high 80s on HumanEval.

[14] *HTMX vs Alpine.js: Lightweight Interactivity 2026*. HTMX (14KB) + Alpine.js (15KB).

### 19.3 Evaluation Datasets NEW

[15] JailbreakBench. (2024). *A Robustness Benchmark for LLM Jailbreaking*. NeurIPS 2024. 500+ jailbreak prompts. Available at jailbreakbench.github.io.

[16] ShareGPT Dataset. (2024). Open-source conversation dataset for false positive testing. Available at huggingface.co/datasets/sharegpt.

[17] CISA Known Exploited Vulnerabilities Catalog. (2025). For Intel validation. Available at cisa.gov/known-exploited-vulnerabilities-catalog.

# 20. Appendices

## Appendix A: Glossary of Terms

| TERM | DEFINITION |
|---|---|
| **ASM** | Attack Surface Management — continuous discovery and monitoring of all internet-facing assets |
| **CPE** | Common Platform Enumeration — standardized naming scheme for software and hardware |
| **CVE** | Common Vulnerabilities and Exposures — publicly disclosed cybersecurity vulnerabilities |
| **CVSS** | Common Vulnerability Scoring System — framework for rating vulnerability severity (0–10) |
| **EPSS** | Exploit Prediction Scoring System — probability score (0–1) of a vulnerability being exploited |
| **HTMX** | Lightweight JavaScript library (14KB) for dynamic UI updates without full page reload |
| **JWT** | JSON Web Token — compact token for securely transmitting information |
| **KEV** | Known Exploited Vulnerabilities — CISA catalog of vulnerabilities actively exploited |
| **LLM** | Large Language Model — AI model trained on vast text data (e.g., GPT, Claude, Qwen) |
| **MTTR** | Mean Time to Remediate — average time to fix a vulnerability |
| **NVD** | National Vulnerability Database — US government repository of CVE data |
| **ONNX** | Open Neural Network Exchange — format for ML model interoperability |
| **ORM** | Object-Relational Mapping — technique for converting data between incompatible systems |
| **Q4_K_M** | Quantization level for LLMs — 4-bit with medium K-means, balancing size and quality |
| **RBAC** | Role-Based Access Control — restricts system access based on user roles |
| **Shadow IT** | IT systems used without explicit organizational approval or oversight |
| **SME** | Small and Medium Enterprise — organizations with 1–250 employees |
| **WSL2** | Windows Subsystem for Linux 2 — native Linux environment on Windows |

## Appendix B: System Requirements

| COMPONENT | MINIMUM | RECOMMENDED |
|---|---|---|
| RAM | 16 GB | 20 GB+ |
| CPU | Intel i5 (8th Gen+) / AMD Ryzen 5 | Intel i5-1035G1 or better |
| Storage | 30 GB free | 60 GB+ free (SSD) |
| GPU | None required | None required |
| OS | Windows 10/11 (with WSL2), macOS, or Linux | Windows 11 with WSL2 |
| Docker | Docker Desktop 4.x+ | Docker Desktop with WSL2 backend |

## Appendix C: Quick Start Commands

```
# 1. Clone the repository
git clone https://github.com/yourusername/zerotrace.git
cd zerotrace

# 2. Create environment file
cp .env.example .env
# Edit .env with your admin password and API keys

# 3. Pull the AI model (Qwen2.5-Coder 7B Q4)
ollama pull qwen2.5-coder:7b-instruct-q4_K_M

# 4. Download ML model weights
python scripts/download_models.py

# 5. Start all services
docker-compose up -d

# 6. Create admin user
python scripts/create_admin.py --username admin --password <your-password>

# 7. Access the dashboard
# Open https://localhost:5000

# 8. Access the API documentation
# Open https://localhost:8000/docs
```

# FINAL WORD

ZeroTrace is not just a student project. It is a prototype for national cyber resilience — built on peer-reviewed research, addressing documented gaps, and deployable on a single laptop.

> *"They leave no trace. Because we found them first."*

Prepared by **Aniruddh Gupta**
ZeroTrace Project — Version 2.0 — 2026